TFG — ADRIÁN JULVE NAVARRO

Análisis del Impacto de los Colores de Producto
en las Emociones del Consumidor

GUÍA DE DEFENSA — SCRIPT 1: RECOLECCIÓN DE DATOS

Documento de apoyo para la defensa ante el tribunal

¿Qué hace el Script 1?

El Script 1 es el encargado de recopilar todos los datos del proyecto desde cero. Su función es ir a tres fuentes de datos distintas en internet, descargar las imágenes de los productos, analizar el color de cada imagen y guardar todos los resultados en un archivo CSV que usará el Script 2.

El script está estructurado en cuatro partes bien diferenciadas:

PARTE 1 — Scraper Mahou San Miguel (Selenium)
PARTE 2 — Scraper Amazon Berkeley Objects (descarga directa S3)
PARTE 3 — Scraper Open Food Facts (API pública)
PARTE 4 — Unificación de los tres datasets en uno solo

Antes de las cuatro partes, hay dos funciones compartidas que usan los tres scrapers: una para descargar imágenes y otra para calcular el color de cada imagen.

---

1. Librerías importadas

Al principio del script se importan todas las herramientas externas que Python necesita para funcionar. Cada una tiene un propósito concreto:

os, time, re, io, gzip, json → Herramientas básicas de Python: manejar archivos, pausas, texto, compresión y JSON
requests → Hacer peticiones a internet para descargar páginas e imágenes
pandas (pd) → Crear y manejar tablas de datos (como Excel desde código)
numpy (np) → Operaciones matemáticas con matrices de píxeles
matplotlib → Crear gráficos (usado en el Script 2, aquí solo se importa)
PIL / Image → Abrir, convertir y guardar imágenes
skimage → Convertir imágenes del espacio de color RGB al espacio CIELAB
tqdm → Mostrar barras de progreso en la consola
selenium + webdriver → Controlar Chrome automáticamente para el scraper de Mahou
webdriver_manager → Gestionar la instalación automática de ChromeDriver

---

2. Configuración general

Después de los imports se definen las rutas de todos los archivos que el script va a generar. Esto se hace al principio para que si el usuario necesita cambiar dónde se guardan los datos, solo tenga que tocar este bloque y no buscar por todo el código.

```
CARPETA     →  Ruta base en el escritorio donde se guarda todo
CSV_MAHOU   →  Archivo CSV con los productos de Mahou
CSV_ABO     →  Archivo CSV con los productos de Amazon
CSV_OFF     →  Archivo CSV con los productos de Open Food Facts
CSV_FINAL   →  CSV combinando los tres anteriores
CSV_LIMPIO  →  CSV limpio listo para el Script 2 (Dataset combinado sin
emociones.csv)
```

También se crea automáticamente la carpeta de gráficos si no existe con os.makedirs(..., exist_ok=True). El exist_ok=True evita que dé error si la carpeta ya existía.

---

3. Funciones de color compartidas

Estas dos funciones las usan los tres scrapers. Se definen una sola vez para no repetir el mismo código en cada scraper.

3.1 descargar_imagen()

Esta función recibe la URL de una imagen, la descarga de internet y la guarda en el disco. Tiene una optimización importante: si la imagen ya estaba descargada, no la vuelve a descargar. Esto es clave porque el script puede tardar horas y si se interrumpe, al relanzarlo no empieza de cero.

1. Comprueba si el archivo ya existe en disco → si sí, devuelve la ruta directamente

2. Hace la petición HTTP con requests.get() con un timeout de 20 segundos
3. Convierte los bytes descargados en una imagen con PIL y la pasa a formato RGB
4. La guarda en disco en formato JPEG con calidad 90 (buena calidad, tamaño razonable)
5. Si algo falla, devuelve None para que el programa siga sin romperse

3.2 calcular_color()

Esta es la función más importante del script desde el punto de vista académico. Recibe la ruta de una imagen y devuelve 7 números que describen cuantitativamente su color. Es el núcleo técnico del trabajo.

¿Cómo funciona?

```
PASO 1 — Abrir la imagen
  Abre el archivo y lo convierte a RGB para asegurarse de que tiene tres canales
  (algunas imágenes vienen en RGBA con canal de transparencia o en escala de grises)

PASO 2 — Convertir a matriz numérica
  np.asarray() convierte la imagen en una tabla de números de tamaño (alto, ancho, 3)
  Donde ese 3 representa los tres canales de color: Rojo, Verde y Azul
  Una imagen de 512x512 píxeles se convierte en una matriz de 512 x 512 x 3 = 786.432 números

PASO 3 — Calcular medias RGB
  arr[:,:,0].mean() coge todos los valores de la capa Roja y calcula su media
  Se repite para Verde (capa 1) y Azul (capa 2)
  Resultado: mean_R, mean_G, mean_B  (valores entre 0 y 255)

PASO 4 — Convertir a espacio CIELAB
  Primero se divide la matriz entre 255 para normalizar los valores a rango [0,1]
  Luego skcolor.rgb2lab() convierte cada píxel de RGB a CIELAB
  CIELAB es un espacio de color diseñado para imitar la percepción humana del color
  Sus tres componentes son: L* (luminosidad), a* (rojo-verde), b* (amarillo-azul)

PASO 5 — Calcular medias y contraste CIELAB
  mean_L: media de la luminosidad de todos los píxeles
  mean_a: media del componente rojo-verde
  mean_b: media del componente amarillo-azul
  contrast_L: desviación típica de L* — mide cuánto varía la luminosidad en la imagen
          Un valor alto indica mucho contraste (zonas oscuras y claras mezcladas)
          Un valor bajo indica imagen uniforme (por ejemplo un fondo blanco liso)
```

La elección del espacio CIELAB está justificada por la literatura científica del TFG. RGB es un espacio técnico diseñado para pantallas, pero CIELAB imita la forma en que el ojo

humano percibe los colores, lo que lo hace adecuado para estudiar las emociones que provocan.

---

4. Scraper Mahou San Miguel — Selenium

Mahou San Miguel es la fuente de datos más compleja porque su web usa JavaScript dinámico, lo que significa que el contenido no está en el HTML estático sino que se genera cuando el navegador ejecuta el código de la página. Por eso no podemos usar requests directamente: necesitamos un navegador real que ejecute ese JavaScript.

La solución es Selenium, una librería que permite controlar Chrome desde Python como si fuera un usuario humano navegando. El navegador se abre en modo invisible (sin ventana visible) para no interrumpir el trabajo del ordenador.

4.1 Configuración de Chrome

Se configura Chrome con varias opciones antes de abrirlo:

```
--headless=new          →  Chrome se abre sin ventana visible, en segundo plano
--no-sandbox            →  Desactiva una capa de seguridad que da problemas en algunos sistemas
--disable-dev-shm-usage →  Evita errores de memoria en entornos restringidos
--window-size=1920,1080 →  Ventana virtual grande para que carguen todos los elementos
user-agent=Mozilla/5.0  →  Chrome se identifica como navegador normal para no ser bloqueado
--lang=es-ES            →  La página se carga en español
```

4.2 Función aceptar_cookies()

Al entrar a la web, aparece el banner de cookies. Esta función lo busca y hace clic automáticamente para poder seguir navegando. Usa WebDriverWait para esperar hasta 6 segundos a que el botón sea clickable, y XPath para encontrarlo por su texto aunque tenga mayúsculas o minúsculas distintas.

4.3 Función extraer_urls_categoria()

Esta función entra en una categoría de la tienda de Mahou y recorre todas sus páginas para recopilar las URLs de los productos. La tienda usa paginación: los productos se muestran de 12 en 12, con el parámetro ?start=0, ?start=12, ?start=24...

1. Construye la URL de la página actual con el parámetro start
2. El navegador carga esa página y espera 3 segundos
3. Ejecuta un scroll hasta el final (JavaScript) para que carguen todos los productos
4. Busca todos los enlaces que contengan /tienda/p/ en la URL (patrón de producto)
5. Añade los nuevos a la lista si no estaban ya

```
6. Avanza a la siguiente página (start += 12)
7. Para si la página tiene menos de 12 productos (última página) o si ya hay 300 URLs
```

4.4 Función extraer_datos_producto()

Una vez tenemos la URL de un producto, esta función entra en su página y extrae el nombre y la URL de su imagen principal. Usa dos estrategias para encontrar la imagen:

```
Estrategia 1 (preferida): busca la etiqueta meta og:image en el HTML
  Las webs modernas incluyen esta etiqueta con la imagen principal de mayor calidad

Estrategia 2 (alternativa): busca directamente una etiqueta <img> con selectores CSS
  Filtra solo imágenes cuya URL contenga 'demandware' o 'mahou' para evitar iconos

En ambos casos se eliminan los parámetros de la URL (lo que va después del ?):
  'foto.jpg?width=200&height=200'  →  'foto.jpg'  (imagen original sin recortar)
```

4.5 Ejecución del scraper Mahou

El bloque principal del scraper ejecuta todo el proceso en tres fases:

```
FASE 1: Abre la tienda, acepta cookies y recorre las 13 categorías del diccionario
      Para cada categoría llama a extraer_urls_categoria() y guarda todas las URLs
      Elimina duplicados comparando las URLs con un set

FASE 2: Para cada URL única, llama a extraer_datos_producto() y guarda nombre e imagen
      Usa tqdm para mostrar barra de progreso
      Al terminar cierra Chrome con driver.quit() para liberar memoria

FASE 3: Descarga cada imagen con descargar_imagen() y calcula su color con calcular_color()
      Guarda los resultados en una lista y la convierte en CSV con pd.DataFrame()
```

---

5. Scraper Amazon Berkeley Objects — S3

Amazon Berkeley Objects (ABO) es un dataset académico público que Amazon publicó para investigación. Contiene información de millones de productos de Amazon con sus imágenes. No hay que hacer scraping de una web porque Amazon los pone disponibles directamente en su servidor S3 (un sistema de almacenamiento en la nube).

Esta fuente aporta la mayor parte del dataset: 7.777 de los 10.480 productos finales. Cubre 80 categorías de producto en moda, hogar, cocina, tecnología, cosmética, juguetes, deporte y mascotas.

5.1 Función descargar_con_cache()

ABO trabaja con archivos grandes comprimidos. Esta función los descarga y los guarda en una carpeta de caché para que si el script se interrumpe y se relanza, no tenga que volver a descargar archivos de varios cientos de MB. Si el archivo ya está en caché, lo lee directamente del disco.

5.2 Funciones auxiliares

```
extraer_nombre()       →  Extrae el nombre del producto del diccionario, priorizando inglés
extraer_product_type() →  Extrae el tipo de producto (SHIRT, LAMP...) en mayúsculas
extraer_marca()        →  Extrae la marca del producto, priorizando inglés
```

5.3 Categorías y mapa de traducción

ABO usa códigos en inglés para identificar los tipos de producto: SHIRT, LAMP, POT... El script define dos estructuras:

```
CATEGORIAS_INTERES  →  Un conjunto (set) con los 80 códigos de categoría que nos interesan
                       Se usa para filtrar: si el tipo de producto no está aquí, se ignora

MAPA_CATEGORIAS     →  Un diccionario que traduce cada código al nombre en español
                       SHIRT → Moda - Camisetas
                       LAMP  → Hogar - Lámparas
                       POT   → Cocina - Ollas
                       ... (80 traducciones en total)
```

5.4 Proceso de extracción — 4 fases

```
FASE 1 — Índice de imágenes
  Descarga images.csv.gz: un índice que relaciona cada image_id con su URL real
  Contiene millones de entradas

FASE 2 — Listings de productos
  Descarga 10 archivos listings_0.json.gz a listings_9.json.gz
  Cada archivo tiene miles de productos en formato JSON, uno por línea
  Para cada producto: extrae el tipo, comprueba si está en CATEGORIAS_INTERES
  Si está y tiene imagen, lo guarda en la lista productos_filtrados
  Para cuando se alcanzan 10.000 productos

FASE 3 — Deduplicación
  Elimina productos con el mismo item_id usando un set
```

```
  Limita la lista al máximo de 10.000

FASE 4 — Descarga y cálculo de color
  Para cada producto construye la URL de su imagen en Amazon (formato _SX512_.jpg)
  Descarga la imagen y calcula sus 7 variables de color
  Guarda todo en CSV
```

---

6. Scraper Open Food Facts — API

Open Food Facts es una base de datos colaborativa y abierta de productos alimentarios, similar a la Wikipedia de la comida. Cualquiera puede consultarla mediante su API pública. Esta fuente aporta 2.622 productos de 30 categorías de alimentación vendidos en España.

A diferencia de Mahou (que requería Selenium) y de Amazon (que tenía archivos para descargar), Open Food Facts tiene una API REST: le enviamos una petición HTTP con unos parámetros y nos responde con los datos en formato JSON directamente. Es la fuente más limpia y sencilla de las tres.

6.1 Función buscar_productos()

Esta función hace las consultas a la API paginando los resultados. La API devuelve los productos de 50 en 50, así que hay que ir pidiendo página a página hasta tener suficientes o hasta que no haya más.

```
Parámetros que se envían a la API en cada petición:
  categories_tags  →  Código de la categoría (ej: en:beers para cervezas)
  countries_tags   →  en:spain para filtrar solo productos vendidos en España
  fields           →  Solo pedimos 7 campos para que la respuesta sea ligera
  page_size        →  50 productos por página
  sort_by          →  unique_scans_n — ordena por los más escaneados (más populares)
  json             →  1 para que responda en formato JSON

Lógica de paginación:
  Pide página 1 → guarda los 50 productos → pide página 2 → guarda → ...
  Para cuando llega al máximo o cuando la API dice que no hay más
  Espera 1 segundo entre páginas para no saturar el servidor
```

6.2 Proceso completo

```
PASO 1 — Para cada categoría del diccionario llama a buscar_productos()
      Reparte el máximo de 10.000 productos equitativamente entre las 30 categorías
```

PASO 2 — Deduplicación por código de barras

Usa el código de barras como identificador único (más fiable que el nombre)

Si no tiene código de barras, usa la URL de la imagen como alternativa

PASO 3 — Descarga de imágenes y cálculo de color

Igual que en Amazon: descarga cada imagen y calcula sus 7 variables de color

Guarda todo en CSV

---

7. Unificación de los tres datasets

Una vez los tres scrapers han generado sus CSV individuales, la función unificar_datasets() los combina en uno solo. Es el paso final del Script 1 y genera el archivo que recibe el Script 2.

PASO 1 — Definir las 11 columnas comunes

Los tres CSVs deben tener exactamente las mismas columnas en el mismo orden

Si alguno tiene columnas extra (Mahou tenía 'marca'), se descartan aquí

PASO 2 — Cargar y apilar

pd.read_csv() carga cada CSV como tabla

df[COLUMNAS] recorta cada tabla a las 11 columnas definidas

pd.concat() las apila una encima de la otra (como pegar tres Excel en uno)

ignore_index=True reinicia la numeración de filas desde 0

PASO 3 — Limpieza básica

dropna(): elimina filas con algún valor de color vacío

drop_duplicates(subset=['imagen_url']): elimina productos con la misma imagen

reset_index(): reordena los números de fila para que queden consecutivos

PASO 4 — Guardar

Guarda el resultado como Dataset combinado sin emociones.csv

Este es exactamente el archivo de entrada del Script 2

---

8. Bloque de ejecución principal

Al final del Script 1 hay un bloque especial con if __name__ == '__main__':. Esta línea garantiza que el código solo se ejecuta cuando lanzas el Script 1 directamente, no si otro script lo importa.

Dentro hay tres interruptores manuales:

```
EJECUTAR_MAHOU = True   # Cambia a False si ya tienes los datos de Mahou
EJECUTAR_ABO   = True   # Cambia a False si ya tienes los datos de Amazon
EJECUTAR_OFF   = True   # Cambia a False si ya tienes los datos de Open Food
Facts
```

Esto es muy útil porque el scraping puede tardar varias horas. Si el Script 1 se interrumpe a medias y los datos de Amazon ya están descargados, puedes poner EJECUTAR_ABO = False y el script no los vuelve a descargar, ahorrando tiempo.

Siempre se ejecutan unificar_datasets() al final, que combina los tres CSVs en el archivo final.

---

9. Resumen ejecutivo para la defensa

Si el tribunal te pregunta '¿qué hace el Script 1?' en una frase:

> El Script 1 recopila automáticamente imágenes de productos de tres fuentes distintas, calcula las variables de color de cada imagen (RGB y CIELAB) y genera un CSV unificado con 10.480 productos listos para ser analizados por el Script 2.

Si te preguntan por qué tres fuentes distintas:

> Mahou San Miguel → Para tener productos del mercado español real con alto valor de marca
> Amazon (ABO) → Para tener diversidad de categorías: moda, hogar, tecnología, etc.
> Open Food Facts → Para tener productos de alimentación con sus envases reales
>
> Las tres juntas dan un dataset variado en categorías, fuentes y tipos de producto, lo que hace el modelo más robusto y generalizable.

Si te preguntan por qué se usa Selenium para Mahou y no para los demás:

> La web de Mahou carga su contenido dinámicamente con JavaScript.
> Si intentas descargar el HTML directamente con requests, los productos no aparecen porque aún no se han generado. Selenium soluciona esto abriendo Chrome real y dejando que ejecute el JavaScript antes de leer el contenido.
>
> Amazon no necesita Selenium porque sus datos están en archivos JSON comprimidos descargables directamente. Open Food Facts tiene una API pública bien documentada.

Si te preguntan por qué se usa CIELAB y no solo RGB:

RGB es un espacio técnico diseñado para representar colores en pantallas.
No refleja bien cómo los percibe el ojo humano: un cambio de 10 puntos en Rojo no tiene el mismo impacto perceptivo que un cambio de 10 puntos en Verde.

CIELAB fue diseñado específicamente para modelar la percepción humana del color.
En CIELAB, una diferencia numérica igual representa una diferencia visual igual independientemente del color. Por eso es el espacio estándar en estudios de psicología del color, incluyendo Gilbert et al. (2016) que es la referencia del TFG.